

Brief: Stripe payment flow

Stripe Payment Processing Flow — Technical Brief

Executive Summary

Stripe delivers a modular yet end-to-end payment stack that combines PCI-compliant payment collection, orchestration logic (PaymentIntent and SetupIntent objects), fraud mitigation (Radar), payout scheduling, and extensible reporting through APIs and the Dashboard. A complete Stripe payment flow typically moves through seven checkpoints: customer data collection, tokenization and PaymentIntent creation, authentication (3D Secure/SCA), authorization, capture/settlement, post-payment lifecycle events (webhooks, refunds, disputes), and payout reconciliation. By leaning on Stripe's opinionated flow, builders get lower PCI scope, global payment method support, and automated compliance updates, but they must still design resilient webhook listeners, idempotent server logic, and a reconciled ledger. This brief details each phase, highlights configuration choices (automatic vs. manual capture, on-session vs. off-session, multi-processor failover), and ends with implementation timelines plus tactical recommendations for Building Shultz's ecosystem. Sources: Stripe Documentation on PaymentIntents, Webhooks, and Radar [Stripe Docs - PaymentIntents](#), NerdWallet overview (2026) [NerdWallet](#), and FitSmallBusiness guide (2024) [FitSmallBusiness](#).

1. Stripe Payment Infrastructure Overview

1.1 Core Components

- **Stripe Dashboard:** Web console to configure payment methods, create API keys, monitor balances, issue refunds, manage disputes, and

export financial data. Offers role-based access control, test/live mode toggles, and granular log inspection.

- **API + SDKs:** RESTful endpoints and client libraries (JS, Python, Ruby, Go, etc.) that expose every object (PaymentIntent, Customer, Product, Price) with idempotent operations. Stripe.js handles PCI compliance by preventing raw card data from hitting merchant servers.
- **Stripe Elements / Checkout:** Pre-built UI layers. Elements is customizable form components; Checkout is a hosted page that handles localization, payment method display, wallet buttons, and SCA.
- **Webhook System:** Stripe asynchronously fires events (e.g., `payment_intent.succeeded`, `charge.refunded`) to merchant endpoints. Webhooks enable final state confirmation, ledger updates, and service downstream notifications even when customers close browsers early.
- **Balance & Payout Layer:** Stripe aggregates captured funds into an available balance, then disburses to linked bank accounts based on payout schedules (daily, weekly, monthly, or manual). Instant payouts available in supported regions with an extra fee.
- **Risk & Compliance:** Radar uses machine learning on Stripe's global network to score transactions, enforce blocklists, and route flagged payments for manual review. Compliance updates (PCI DSS, PSD2/SCA, NACHA, etc.) are handled centrally.

1.2 Mode Separation

- **Test Mode:** Uses separate API keys, webhooks, and data. Ideal for staging flows without touching real money. Stripe provides extensive test cards to simulate outcomes (insufficient funds, authentication required, etc.).
 - **Live Mode:** Production credentials that move money. Merchants must segregate keys, store them securely (e.g., environment variables or secrets manager), and audit usage regularly.
-

2. End-to-End Payment Flow (Card + Wallet Example)

Stripe's flagship card flow is orchestrated by **PaymentIntent**, an object representing a customer's intent to pay a specific amount. The

PaymentIntent evolves through statuses (`requires_payment_method` , `requires_confirmation` , `requires_action` , `processing` , `succeeded` , etc.). Below is a canonical seven-step journey.

Step 1: Customer Initiation & Data Capture

1. Customer selects a product/plan and provides billing details on the merchant app.
2. Frontend uses **Stripe Elements** or direct Stripe.js integration to tokenize sensitive payment data, yielding a `payment_method` ID. Merchants never store raw PAN/CSC, reducing PCI scope to SAQ A.
3. Optional: Collect customer email, shipping address, tax IDs, and metadata for downstream analytics.

Step 2: Create/Update PaymentIntent (Server-Side)

1. Backend receives `payment_method` ID via HTTPS.
2. Server calls `POST /v1/payment_intents` (or updates existing one) with amount, currency, capture method, confirmation method, and `payment_method` .
3. Stripe immediately runs validation (currency, amount limits) and attaches the payment method. Status becomes `requires_confirmation` .

Step 3: Confirm PaymentIntent

1. Server or client calls `confirm` endpoint, depending on integration style:
2. **Automatic confirmation:** Client uses `stripe.confirmCardPayment(clientSecret)` so front-end can handle dynamic authentication challenges.
3. **Manual confirmation:** Backend confirms, suitable for server-driven flows.
4. If 3D Secure or regulatory SCA is required, PaymentIntent moves to `requires_action` ; Stripe returns instructions for handling the challenge via Stripe.js. Otherwise it proceeds to authorization.

Step 4: Authorization & Fraud Checks

1. Stripe requests authorization from card networks/banks; funds are reserved but not yet captured.
2. Radar scores the payment. Depending on rules, Stripe can auto-block, require manual review, or proceed.
3. Status transitions to `processing` or `requires_capture` (if manual capture is enabled).

Step 5: Capture & Settlement

1. **Automatic capture** (default) captures funds immediately after authorization.
2. **Manual capture** gives merchants up to 7 days (card) to capture via `POST /v1/payment_intents/{id}/capture`. Useful for verifying inventory or partial fulfillment.
3. Upon capture, Stripe creates a `Charge` object, moves `PaymentIntent` to `succeeded`, and schedules funds for payout.

Step 6: Post-Payment Lifecycle

1. **Webhooks:** Merchant receives `payment_intent.succeeded` to trigger order fulfillment, send receipts, and update internal ledgers. Webhooks ensure consistency even if the client disconnects.
2. **Refunds:** Initiated via Dashboard or API (`POST /v1/refunds`). Partial refunds reference the original `Charge`.
3. **Disputes:** Cardholders can dispute via their bank. Stripe notifies the merchant (`charge.dispute.created`), debits the balance for the disputed amount, and expects evidence uploads before a deadline.
4. **Reconciliation:** Combine Stripe Balance reports with accounting software. Stripe can push to accounting platforms via APIs or data exports.

Step 7: Payouts

1. Captured funds appear in the **available balance** after settlement times (typically T+2 for U.S. cards).

2. Payouts follow the configured schedule to the merchant's bank account. Instant payout (if enabled) moves funds within minutes for an extra percentage fee.
 3. Payout failure notifications allow merchants to fix bank details and retry.
-

3. Variations in the Flow

3.1 Alternative Payment Methods

Stripe PaymentIntents abstract payment method differences. Key variations:

- **Wallets (Apple Pay, Google Pay):** Use the same PaymentIntent but leverage platform-specific tokens. Reduced friction on mobile.
- **ACH Debit:** Requires bank account verification via microdeposits or Plaid. Settlement takes longer (2-5 days) and includes return risk.
- **Buy Now Pay Later (Affirm, Klarna):** PaymentIntent interacts with third-party lenders; merchants receive funds upfront, customers pay over time.
- **PayPal/Venmo:** Through Stripe's new partnerships (region-dependent), still orchestrated via PaymentIntent.

3.2 Subscriptions & Recurring Billing

- Use **Stripe Billing** with `Product`, `Price`, and `Subscription` objects.
- First payment uses PaymentIntent; subsequent renewals use `SetupIntent` captured payment methods for off-session charges.
- Stripe handles dunning via Smart Retries, email reminders, and card updater services.

3.3 In-Person (Stripe Terminal)

- Terminal devices pair with Stripe readers. Payment-intent-like flow occurs but with reader SDKs.
 - Ideal for pop-up shops, events, or hybrid online/offline businesses.
-

4. Security, Compliance, and Risk Management

4.1 PCI and Data Security

- Using Stripe Elements/Checkout keeps merchants at **PCI DSS SAQ A** scope. No need to manage encryption or tokenization.
- For custom UI without Elements, merchants must meet SAQ A-EP or SAQ D—significantly higher compliance burden.
- Stripe rotates certificates, encrypts data, and provides audit logs.

4.2 Authentication & Regulatory Requirements

- **PSD2/SCA** (EU/UK): Stripe handles PSD2 logic automatically. Merchants can create rules to prefer exemptions (TRA, MIT) but must provide documentation.
- **3D Secure (3DS)**: Stripe triggers when required. Merchants can configure Radar rules to request 3DS above certain amounts.

4.3 Radar & Custom Rules

- Default Radar leverages network-wide data.
 - Merchants can define allow/deny lists, velocity checks, and custom scoring rules.
 - Review flows integrate into Dashboard for manual adjudication.
-

5. Operational Monitoring & Reporting

5.1 Dashboards and Alerts

- **Payments Overview**: Monitor volume, conversion, average ticket.
- **Balance Tab**: View pending vs. available balances, payouts, deductions.
- **Radar**: Track block/allow rates, rule effectiveness.
- **Alerts**: Configure email/SMS/Slack notifications for failed webhooks, payout failures, dispute deadlines.

5.2 Reconciliation

- Stripe provides **Payout Reconciliation report** aligning charges, refunds, disputes, fees per payout.
 - Export CSVs or integrate with accounting suites (QuickBooks, Xero).
 - Recommended: maintain an internal ledger keyed by PaymentIntent ID to align product fulfillment and financial entries.
-

6. Actionable Recommendations for Building Shultz

#	Recommendation	Owner	Rationale	Timeline
1	Implement Stripe Checkout for initial launch, then graduate to Elements for custom branding	Sol (integration) + Jed (UX feedback)	Checkout accelerates compliance and SCA handling; Elements later enables signature look	Week 1-2
2	Leverage PaymentIntents with manual capture for high-ticket commissions (custom builds)	Sol (backend)	Allows verification of materials/availability before capturing funds	Design Week 2, deploy Week 4
3	Stand up dedicated webhook microservice (e.g., on VPS) with idempotent handlers and retries	Sol	Ensures order management stays in sync; Stripe retries for 3 days, but you need ACK logging	Week 1 build, Week 2 smoke test

#	Recommendation	Owner	Rationale	Timeline
4	Configure Radar custom rules (e.g., block mismatched country/IP, require 3DS above \$1,000)	Sol	Reduces fraud exposure for bespoke products	Week 3
5	Automate payout reconciliation by pushing Stripe data into accounting ledger nightly	Sol + CPA partner	Prevents month-end scramble and supports tax filings	Week 4 integration, ongoing
6	Set up dispute playbook: template evidence packet, assign owner, track deadlines	Sol	Minimizes lost disputes and revenue leakage	Week 5
7	Use Checkout Sessions + Stripe Tax to auto-calculate EU/VAT as soon as digital goods launch	Sol	Keeps compliance ready for future eBooks/courses	Week 6

7. Implementation Timeline (6-Week Plan)

1. Week 1

- Obtain API keys, restrict them via dashboard roles.
- Build sandbox environment with Stripe Checkout (test mode).
- Stand up webhook endpoint with logging, signature verification.

5. Week 2

- Integrate PaymentIntent manual capture path for flagship products.

7. Add client-side validation (Elements) for improved UX.
 8. Begin Radar baseline (observe default scoring).
 9. **Week 3**
 10. Define Radar custom rules & allowlists/denylists.
 11. Configure email/SMS alerts for failed payouts and disputes.
 12. Start drafting dispute evidence templates.
 13. **Week 4**
 14. Connect accounting stack for automatic reconciliation.
 15. Enable payout schedule review (evaluate instant payout need).
 16. Conduct failure-mode tests (webhook downtime, partial refunds).
 17. **Week 5**
 18. Document SOPs for refunds, disputes, payout reconciliation.
 19. Enable live mode with limited product set; monitor conversion.
 20. **Week 6**
 21. Expand payment methods (ACH, wallets).
 22. Add Stripe Tax/Checkout localization for global audience.
 23. Review metrics, adjust Radar rules and capture strategies.
-

8. Next Steps Checklist

- [] Generate restricted API keys (publishable + secret) and store via environment variables.
 - [] Implement server-side PaymentIntent endpoints with idempotency keys.
 - [] Deploy webhook receiver with queue-based retry handling.
 - [] Configure Stripe Checkout session flow; gather branding assets.
 - [] Map physical fulfillment pipeline to webhook events (order creation, shipping).
 - [] Draft refund/dispute SOPs with responsible owners and deadlines.
 - [] Schedule Day-30 review to evaluate payment method expansion and subscription readiness.
-

9. Takeaways

- Stripe's PaymentIntent-centric architecture ensures compliance with SCA, supports multi-method payments, and reduces PCI exposure, but it requires disciplined backend orchestration.
 - Radar and webhooks are not optional—they're the backbone for fraud control and financial reconciliation.
 - A phased rollout (Checkout → Elements → custom flows) lets Building Shultz launch quickly while planning for branded experiences and complex products like IronEdit subscriptions.
 - Establishing a six-week implementation plan with clear owners and SOPs guarantees operational readiness, supports Jed's "Building What Matters" ethos, and keeps the march toward a million-dollar business on rails.
-

Sources

1. Stripe Docs — PaymentIntents: lifecycle, confirmations, capture options (<https://stripe.com/docs/payments/payment-intents>)
2. Stripe Docs — Webhooks: reliability, signature verification (<https://stripe.com/docs/webhooks>)
3. Stripe Docs — Radar for Fraud Teams (<https://stripe.com/docs/radar>)
4. NerdWallet — "What Is Stripe, and How Does It Work to Accept Payments?" (March 20, 2026) (<https://www.nerdwallet.com/business/software/learn/what-is-stripe>)
5. FitSmallBusiness — "What Is Stripe and How Does it Work? Small Businesses Guide" (Oct 30, 2024) (<https://fitsmallbusiness.com/what-is-stripe/>)